

BEC305 Python Programming — Assignment 4 — Programmer-defined types and object semantics

Module 4: Classes and Objects

Field	Value
Maximum marks	20
Weightage	10% of CIE
Issue date	<<TBD>>
Due date	<<TBD>>
Items	10 problems + 3 theory questions
CO / Bloom coverage	CO4: L3 · L1–L2 0, L3–L4 6, L5–L6 7

Objective

Make object semantics concrete — aliasing, copying depth, purity versus modification — and develop judgement about when a class is the right structure and when it is ceremony.

Learning outcomes

- Student will be able to define classes with `__init__`, methods and special methods.
- Student will be able to predict whether a function modifies its argument and justify the design either way.
- Student will be able to choose the correct copy depth and verify independence with the `is` operator.
- Student will be able to justify a choice between a class, a dataclass, a namedtuple and a dictionary.

Part A — Problems (17 marks)

P1. Define a `Point` class with no `__init__` and assign its attributes by hand. Deliberately misspell one attribute, then read the intended one, and record exactly where and when the error surfaced relative to the mistake. [C04 | L4 | 2 marks]

P2. Define a `Rectangle` that holds a `Point` as an attribute. Implement `find_center` as a pure function returning a new `Point`, and demonstrate that it composes directly into another function taking a `Point`. [C04 | L3 | 2 marks]

P3. Write `grow_rectangle` as a modifier and `grown_rectangle` as a pure function producing the same result. Compare their call sites, their tests, and what each allows the caller to assume. [C04 | L4 | 2 marks]

P4. Demonstrate plain assignment, `copy.copy` and `copy.deepcopy` on a `Rectangle` holding a `Point`. Present a table of is comparisons for both the outer object and the embedded `Point` after each. [C04 | L4 | 2 marks]

P5. Implement a `Time` class and add two `Times` twice: once by attribute-by-attribute arithmetic with carrying patches, once by conversion to seconds and back. Count the branches in each and test both on a case that rolls the hour over. [C04 | L5 | 2 marks]

- P6.** Add `__init__` with defaults to your `Time` class and demonstrate instantiation with zero, one, two and three arguments. Then show the mutable-default bug on a class holding a list, and fix it. [C04 | L4 | 2 marks]
- P7.** Define `__str__` and `__repr__` on `Time`. Show the output of `print(t)`, `str(t)`, `print([t])` and evaluating `t` at the interactive prompt, explaining which method each used. [C04 | L4 | 2 marks]
- P8.** Overload `+` on `Time` to accept both another `Time` and an integer number of seconds using `isinstance` dispatch. Then sketch how the same capability would be provided polymorphically and state which you would ship. [C04 | L5 | 1 marks]
- P9.** Define `__eq__` on a class, demonstrate that instances can no longer be dictionary keys, and fix it two different ways. [C04 | L5 | 1 marks]
- P10.** Take one data structure from your own work and implement it four ways — tuple, `namedtuple`, `dataclass` and full class — then argue for one, citing the operations the program actually performs. [C04 | L5 | 1 marks]

Part B — Theory (3 marks)

- T1.** Explain why Python creates attributes on assignment rather than requiring declaration, naming one benefit and one cost of that choice. [C04 | L5 | 1 marks]
- T2.** Argue for or against modifiers, given that a pure function is easier to test but allocates a new object per call. State the conditions under which you would accept a modifier. [C04 | L5 | 1 marks]
- T3.** Explain why operator overloading should be reserved for conventional meanings, using an example of an overload that would mislead a reader. [C04 | L5 | 1 marks]

Submission guidelines

- Format: PDF, single file
- File name: `<USN>_BEC305_Assignment4.pdf`
- Channel: LMS upload
- Late policy: 10% per day, nothing after 5 days
- Every numerical answer must carry units and a stated reference direction.

Assessment rubric

Criterion	Weight	Excellent	Good	Satisfactory	Needs improvement
Class definition and initialisation	6	Classes defined with <code>__init__</code> , sensible defaults and stated invariants; no required attribute assigned outside <code>__init__</code> ; mutable-default bug demonstrated and fixed.	Classes correct with minor gaps in defaults or invariants.	Classes work but attributes are assigned ad hoc after construction.	Missing <code>self</code> , or instances left in half-built states.

Criterion	Weight	Excellent	Good	Satisfactory	Needs improvement
Object semantics: aliasing and copying	5	Every case demonstrated with is comparisons on both outer and embedded objects, and the correct copy depth chosen with a stated reason.	Correct demonstrations with one gap in the reasoning.	Copies made but independence not verified.	Shallow copy assumed independent, or is and == confused.
Purity, modifiers and design	5	Each function explicitly labelled pure or modifier, named and documented accordingly, with the canonical-form redesign implemented and its branch count compared.	Distinction observed with adequate justification.	Functions written without stating which kind they are.	Functions that both return a value and mutate an argument.
Special methods and structure choice	4	<code>__str__</code> , <code>__repr__</code> , <code>__add__</code> and <code>__eq__</code> implemented correctly with the hashability consequence handled; the four-way structure comparison argued from actual operations.	Special methods correct with a brief structural argument.	Special methods implemented without discussion of the trade-offs.	<code>__str__</code> prints instead of returning, or a full class written where only <code>__init__</code> exists.
Total	20				

Course outcome attainment

Course outcome	Marks carried
CO4	20
Total	20

Academic integrity

Individual submission. Identical submissions score zero for both parties.